

CSE 375/475 - Fall 2026
Assignment 1: TBB for Parallelism
Due date: Friday October 2

Part 1: Exploring the power of using TBB

The purpose of this part is to assess the efficiency of TBB. The code snippet given in the slides implements a naive $O(n)$ prime checker for integers $1 \dots N$. We are not going to try to change the algorithm to improve performance. Instead, we will try to parallelize it using three different approaches:

1. TBB `parallel_for`.
2. Manually using `std::thread` with static load balancing.
3. Manually using `std::thread` with dynamic load balancing.

Keep in mind that for small problems, it's likely that different approaches perform similarly. But for $N = 10^9$, it is possible to get a HUGE speedup and the differences between approaches should arise.

The goal is to analyze how different approaches could be "efficient" and "hard to implement". That is why it is important to report "all" your trials (or at least those that make sense). For example, report the performance of the straightforward static load balancing approach that we discussed in lecture.

You must produce a 3-4 page write-up of your experience. Describe the techniques you used to parallelize the code. Include graphs showing the performance for $N = 10^3$, 10^6 and 10^9 , with threads on the X axis and time on the Y axis. Results should be the average of 5 trials and should discuss variance.

Part 2: parallelizing k-means

The purpose of this assignment is to parallelize an existing sequential program using Intel Threading Building Blocks (TBB). If you have a strong reason to use other models (such as OpenMP, OpenCilk, CilkPlus, OpenCL) you need to get my approval first.

The provided program is a serial implementation of the well-known k-means algorithm for clustering. It's a simple implementation, which is borrowed, with few customizations, from this repository: <https://github.com/marcoscastro/kmeans>.

Two datasets are also included to test your application. The provided synthetic datasets might not represent the best candidate to show the effectiveness of your parallel code, but are good starting point to verify that your code is functioning. By searching online you can find many repositories for k-means datasets that can be used for further testing and performance evaluation. For example, a known repository is available at: <http://archive.ics.uci.edu/ml/index.php>. The selection of the dataset is up to you. Each of the available dataset has particular characteristics. Since only you can know where parallelization can be exploited in your code, select your dataset accordingly. Different datasets might have different formats; consider adapting them to fit your application (or change the way your application reads an input dataset). Do not forget that your sequential application should also be tested under the same dataset for comparison.

Your goal is to get the best speedup you can achieve over the provided program by exploiting parallelism and other techniques discussed in class. Keep in mind that performance improvement might also come from using advanced instructions (such as SIMD), fine-tuning important runtime parameters (e.g., number of threads, affinity, chunk sizes), and redesigning part of the solution to break code dependency and increase parallelism.

The final note is about concurrency. A parallel program with race condition is INCORRECT, even if it passes all the tests. The easiest way that to avoid data races, is to use parallelism in way that does not create any flow or data dependency. If you discover that more complex concurrency with carried dependencies can help achieve a better speedup, feel free to explore it ONLY AFTER you implement and test a parallelized version without such dependencies.

The best way to make progress in this project is to first get familiar with the K-means problem itself and its algorithmic solution, then to analyze the potential opportunities for parallelism (with and without carried dependencies), and finally to think of more implementation-detailed optimizations. The textbook has a chapter explaining k-means and provides snippets of code, which can be a good start.

Important notes about submission and grading (both parts)

- It is REQUIRED to use the machines available at sunlab.cse.lehigh.edu. An evaluation performed on your own laptop will NOT BE considered sufficient unless a specific authorization has been granted by me.
- In order to achieve speedup, it is important to check that the machine you are logged in is not overcommitted. The sunlab machines can run 16 threads simultaneously ("less /proc/cpuinfo"), therefore if some other user is running along with you, you might not see the expected performance improvement. Use "top" (or alternative ways) to check the commitment of the machine you are using. If overcommitted, move to another machine. Remember that the "users" command shows the users currently logged in to the machine.
- The submission should be done through coursesite.lehigh.edu and should consist of:
 - The source code of your assignment
 - A document of 3-4 pages per part that summarizes your solution, evaluation, and analysis.
- Assignments will be graded during a one-on-one meeting with each student. You must be able to explain everything in your submission, including design attempts, plots, and analysis.
- The following steps should be sufficient to install TBB using the provided binary files. Instructions are not for the latest version; if you want to go with that, just apply the same instructions to a newer binary.
 - Download from here: <https://github.com/oneapi-src/oneTBB/releases>
---> wget
<https://github.com/uxlfoundation/oneTBB/releases/download/v2022.0.0/oneapi-tbb-2022.0.0-lin.tgz>
---> tar -xvzf oneapi-tbb-2022.0.0-lin.tgz
---> cd oneapi-tbb-2022.0.0/env
---> source vars.sh
The last command has to be executed every time you open a new terminal.
 - Link with tbb using ``-ltbb``
- Before parallelizing k-means, make sure that you can run and test the given serial version. The provided application loads the whole dataset at the beginning of the execution. Once compiled (and the binary "kmeans" is produced), I use the following command line: `cat datasets/XXXX.txt | ./kmeans`